Bottlenecks in increasing network speeds

Tariq Toukan
Mellanox Technologies, Ltd.
tariqt@mellanox.com

Abstract

Network Interface Cards (NIC) speeds are reaching new heights: 100Gbps has been available for a few years, state-of-the-art offer 200Gbps, and next generation will be 400Gbps. These high NIC speeds come with a variety of new challenges and expose bottlenecks in several components. A network's performance is limited by its weaker SW/HW layer or component, and hence bottleneck analysis and addressing is essential to fully utilize the NICs linerate.
In this paper, I review several interesting bottlenecks from different fields, and discuss a set of innovative solutions.

1 Introduction

Bottleneck analysis is the key stage that opens the way to deep discussions that end up with plans of innovative features. In the past two years I invested significant time analyzing the network stack readiness for supporting 200Gbps NIC speeds, and was involved in presenting, discussing, and implementing performance features, in driver, network stack, and memory management subsystem.

Of course, I didn't have a 200Gbps NIC at the time, and needed to find ways to simulate the future behavior. A simple yet powerful technique I used is working with half-size MTU on top of a 100Gbps NIC. Running bandwidth test over this simple configuration immediately exposed a serious bottleneck: the page allocator does not stand the allocation pace.

The page allocator design, components, behavior and performance limitations are presented in section 2, along with the suggested solutions to release the bottleneck.

In section 3, I refer to the PCI Express 3.0 throughput bottleneck. I present innovative HW features that reduce the HW/SW communication overhead and improve the PCI utilization by ideas the like: combining similar descriptors, bulking, etc...

2 Page allocator

2.1 The Linux Buddy allocator

The kernel often needs contiguous memory areas that span multiple page frames, so when serving memory allocation requests, it must minimize external fragmentation. This is the task of the buddy allocator. The concept of the buddy allocator is to maintain direct-mapped table for memory blocks of various orders. Linux uses a separate buddy allocator in each zone Concurrent accesses from different cores to the zone's buddy allocator are synchronized by a zone spinlock:

```
struct zone {
...
    /* Primarily protects free_area */
    spinlock_t lock;
...
};
```

Frequent concurrent accesses to this shared resource would highly congest the lock and limit scalability. To reduce the lock contention, the kernel provides a per-CPU layer of page lists:

```
struct per_cpu_pages;
```

Upon a page allocation request, the kernel approaches the core's PCP. If no page is available, the PCP asks for a batch of pages from the buddy system, under the zone lock. The page release flow works in a symmetric way: a page is returned to the PCP, if the number of pages reaches some (configurable) high threshold, it frees a batch of pages back to the buddy system, under the zone lock.

2.2 Bottleneck description

As long as the PCPs fairly satisfy the page allocation and free operations of their respective cores, they successfully prevent contention on the shared zone lock. But this is not always the case.

In a typical network stack flow, the NIC driver allocates pages per each of its receive rings on a separate core, draining its local PCP. It wraps every received packet with a proper `struct sk_buff *skb` and passes it up the stack. Once the `skb` processing finishes, its pages are released to the local core's PCP.

Stack processing contains several scheduling points, and the handling of an `skb` could move to a core different than the one it originates from, resulting in an imbalanced number of page alloc/free operations on a PCP. Consistent imbalances would cause PCPs to continuously look to to acquire the zone lock and address the buddy allocator: A drained PCP would keep asking for more pages, while an over-satisfied one would keep returning pages to the buddy system. This behavior pattern implies high contention on the zone lock and severely impacts the performance: the higher the packet rate is, and the larger the scale is, the higher the contention.

2.3 Ideas and Solutions

2.3.1 Page reuse

A fundamental technique to reduce the page allocation rate is to better utilize the allocated pages. For default MTU of 1500Bytes and `PAGE_SIZE` of 4KB, each page could be used twice, cutting the allocation rate by half. This technique is implemented in mlx5e driver since kernel v4.13.

An advanced HW feature (Striding Receive WQ) that improves both memory and PCI utilization is presented in section 3.2.3 of this paper.

2.3.2 Page recycle

The general idea is to have the device driver first try recycling its own allocated pages, instead of releasing them and ask for new ones. This is usually done by elevating the page reference counter upon sending it through the network stack, and track the page in a waiting-queue. These outstanding pages can be recycled once their processing is "done", indicated by having their reference counter back to 1. This approach lacks a signaling/callback mechanism to catch the refcount transition from 2 back to 1, which implies that the driver must keep observing the refcnt every time it needs to allocate a page.

However, in many cases this turns out to being very helpful, especially when the system admins know the traffic pattern and the expected processing time, then tune the waiting-queue size accordingly. Feature is implemented in mlx5e driver since kernel v4.9. The work to add this support to the generic `page_pool` API [2] is progressing in an increasing pace, and is expected to be completed very soon.

2.3.3 Remote page release

An intriguing idea that focuses on preventing the PCP imbalances described above is, use a page-return hook to directly release the pages back to the data-structure of their originating core.

The initial idea suggests saving some minimal amount of metadata (the originating core number) in the `skb` structure, and release its pages back to that core's PCP.

The recent direction is to integrate the idea into the `page_pool` API instead, so a pointer to the originating `page_pool` structure is saved and used in the release stage. Two main return hooks are proposed and actively discussed [1]:

1. Page alloactor layer via `put_page`.
2. Network stack layer via `__kfree_skb()`.

2.3.4 Zone lock critical section improvements

Orthogonal efforts were done in the Memory Management (MM) subsystem [3] (to which the page-allocator component belongs) and an RFC was submitted upstream. The RFC profiles the most time consuming parts under zone lock, and suggests changes that reduces the lock contention, and the amount of work done under it.

The key idea is to avoid touching pages `struct page` under the global zone lock, because cache miss on the struct is the most time consuming part under the lock. That's why the RFC attempts to not do merge operations on free path as it causes multiple `struct page`'s to be fetched under the zone lock. Also, not doing merge on free path opens the opportunity of doing cluster allocation on allocation path, i.e. allocate a batch of pages under zone lock without needing to touch every single `struct page` of them. This makes the system favour order-0 alloc/free, perfectly matching the standard buffer size used in the networking stack.

As the page allocator subsystem serves multiple components and subsystems, not only the network stack, the performance implications of the suggested RFC for use cases are to be carefully investigated, discussed, prioritized and addressed, to stage this RFC to the master upstream branch.

3 PCI bottleneck

3.1 Bottleneck description

PCI Express 3.0 x16 width link throughput is 126.03 Gbps. This doesn't leave enough room for a 100Gbps NIC to reach linerate while preserving the typical descriptors layout for RX, TX, and completions. The smaller the packets size, the higher the descriptors overhead, and the severer the problem. This demands new, innovative, bits-economic ways to describing the RX and TX traffic.

3.2 HW features for improved PCI utilization

The heart of the features below is the following idea: "You already use some moderation and do operations in bulks, take advantage of this and reduce the descriptors overhead!".

3.2.1 CQE compression

Reduce PCI overhead by reducing the data to CQEs ratio. Instead of writing full CQEs to memory, multiple almost identical CQEs are merged and compressed. Information that is shared between the CQEs is written once regardless of the number of compressed CQEs. In addition, only the unique information (small amount of bytes compared to full CQE size) is written per CQE.

CQE Compressing reduces PCI overhead by coalescing and compressing multiple CQEs into a single merged CQE. Successful compressing improves message rate especially for small packet traffic.

By enabling the feature, the `XDP_DROP` packet rate of 64B UDP packets increases from 85M PPS to 130M PPS.

Feature is supported by Mellanox NICs ConnectX-4 and newer. Starting from kernel v4.7.

3.2.2 TX descriptor bulking

Multi packet WQE: enables sending multiple various sized packets using a single WQE, where each data segment in the posted WQE points to a separate packet.

This significantly reduces the descriptors overhead when transmission is requested in bulks, as done in `XDP_TX` and `XDP_REDIRECT` actions.

By adding support of the feature, the `XDP_TX` packet rate of 64B UDP packets increases from 70M PPS to 110M PPS.

Feature is supported by Mellanox NICs ConnectX-5 and newer. Starting from kernel v5.0.

3.2.3 RX descriptor bulking

Striding Receive WQ: The legacy Receive Work Queue scheme has the following two inefficiencies:

1. Receive buffers memory utilization. Incoming messages that are smaller than the maximum possible message size leave receive buffers memory unutilized. The bigger the difference between the maximum and minimum message size the more severe this in-efficiency becomes.

2. Receive WQE fetch per incoming packet. A receive WQE must be fetched from host memory (unless already cached on the device) per incoming packet.

The Striding Receive WQ is intended to resolve these legacy Receive WQ inefficiencies. It is achieved by replacing the legacy message-level buffering scheme by a byte-level buffering scheme.

Feature is supported by Mellanox NICs ConnectX-4 LX and newer. Starting from kernel v4.7. [4]

4 Availability

The Mellanox NIC driver (mlx5e) is part of the Linux kernel source, found under the following directory: `drivers/net/ethernet/mellanox/mlx5/core/`

References

[1] BROUER, J. D. https://github.com/xdp-project/xdp-project/tree/master/areas/mem.

[2] BROUER, J. D. Generic page-pool recycle facility? http://people.netfilter.org/hawk/presentations/MM-summit2016/generic_page_pool_mm_summit2016.pdf, 2016.

[3] LU, A. Zone lock RFC v3. https://github.com/aaronlu/linux/tree/zone_lock_rfc_v3.

[4] TOUKAN, T. net/mlx5e: Support rx multi-packet wqe (striding rq). https://www.spinics.net/lists/netdev/msg374052.html.